Advanced Programming - HPC

Tool : Debugger

Monojyotsna Koch

Date : 09.10.26

Agenda and learning objectives

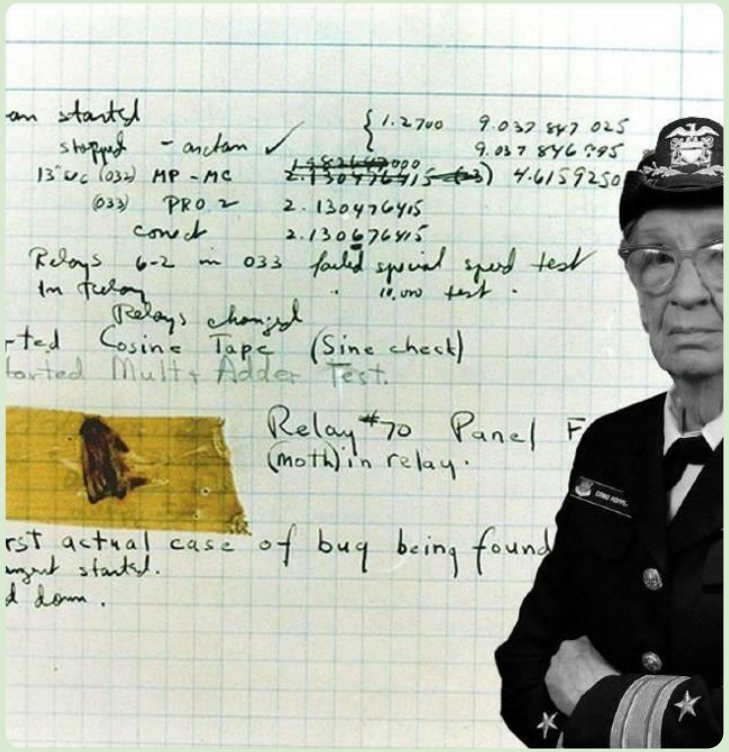
Agenda

1. What is debugging
2. GDB fundamentals
3. Intermediate GDB
4. Parallel debugging
5. GPU Debugging

Objectives

- Follow a debugging workflow
- Compile with -g -O0 and use GDB
- Diagnose programs using GDB
- Inspect OpenMP threads
- GPU debugging tools

The first computer bug



9 September 1947

A moth was found in relay #70, Panel F, of the Harvard Mark II. The logbook entry reads "First actual case of bug being found".

Grace Hopper helped popularise the story. Engineers were already calling faults bugs well before 1947.

What is Debugging?

What's wrong here?

```c
int main()
{
  int a = 10, b = 0;
  int c = a / b;
  return 0;
}
```

What's wrong here?

```c
int main()
{
  int a = 10, b = 0;
  int c = a / b;
  return 0;
}
```

Integer division by zero

It compiles cleanly, then dies at run time with SIGFPE (Floating point exception).

What is debugging?

Finding, analysing and fixing errors in a program, so software is correct, reliable and efficient.

Bug type	Example	Typical tool
Syntax	Missing semicolon	Compiler
Logic	Off-by-one loop bound	GDB, printf
Runtime	int c = a / b with b = 0	GDB, core dump
Memory	Out-of-bounds, use after free	ASan, Valgrind
Concurrency	Data race, deadlock	TSan, GDB threads
Numerical	Integer overflow	Watchpoints, UBSan

Why debugging matters in HPC

Expensive

A job that fails after hours in the queue wastes core-hours.

Scale-dependent

Some bugs appear only with many threads, ranks or large inputs.

Silent

A wrong result from a program that runs fine can corrupt science.

A debugging workflow





GDB

The GNU Project Debugger

GDB: the debugger of the GNU Project

GNU Debugger (GDB)

A powerful open-source debugging tool for C, C++, Fortran and more. It lets you examine the internal workings of a program during execution.

Breakpoints

Stop execution at specific points.

Variable inspection

Examine or modify variable values.

Call stack analysis

Understand the sequence of function calls.

Threads and parallel

Multithreading and parallel debugging support.

Why GDB is crucial: it fits into normal development workflows, so bugs are resolved faster.

Using GDB, step by step

1. Compile

```
gcc -g -O0 -o code code.c
```

-g writes the symbol information GDB needs.

2. Start

```
gdb ./code
(gdb) run
```

Run to the end, or to a breakpoint.

3. Set breakpoints

```
(gdb) break file.c:6
(gdb) break my_func
```

Stop at a line or function start.

4. Step and inspect

```
(gdb) next / step / list
(gdb) print var
(gdb) continue
(gdb) backtrace
```

DWARF Debugging Format (-g)

DWARF (**D**ebugging **W**ith **A**ttributed **R**ecord **F**ormats) is a standardized debugging data format. It's the "map" a compiler gives to a debugger (like GDB or LLDB) to navigate and understand a compiled program.

This map links the low-level machine code back to the high-level source code, enabling features like:

- Setting breakpoints on specific source lines.
- Inspecting the values of variables by name.
- Showing stack traces with function names.
- Stepping through code line by line.

Essential GDB commands

```
gcc -g -O0 -o prog prog.c   then   gdb ./prog
```

Command	Purpose
break file.c:6 / break func	Stop at a line or function
run [args]	Start the program
next / step	Step over / into a call
list	Show source code
print var / info locals	Inspect values
backtrace, up, down, frame N	Walk the call stack
finish	Run until the function returns
watch var	Stop when a value changes
continue / quit	Resume / exit

Demo: hello

```cpp
int main() {
  int j = 3;
  int k = 7;
  j += k;
  k = j * 2;
  std::cout << "Hello everyone!!" << std::endl;
}
```

After the two additions, j is 10 and k is 20. Try set var j = 5.

Try in GDB

```
g++ -g -O0 hello.cpp -o hello
gdb ./hello
(gdb) break main
(gdb) run
(gdb) next
(gdb) print j
(gdb) print k
(gdb) set j=5
(gdb) info breakpoints
(gdb) list
(gdb) watch j
(gdb) info locals
(gdb) info registers
(gdb) quit
```

Demo: segmentation fault

```c
int *sophisticated(int *p) { return NULL; }
int *complicated(int *p) { return p; }
void crash(int *i) { *i = 1; }
void f(int *i) {
  int *j = i;
  j = sophisticated(j);
  j = complicated(j);
  crash(j);
}
int main() { int i; f(&i); return 0; }
```

Try in GDB

```
(gdb) run
(gdb) backtrace
(gdb) frame 1 / frame N(N=0,1,2)
(gdb) print j
(gdb) watch j
(gdb) break f
(gdb) continue
(gdb) quit
```

j is 0x0 because sophisticated() returned NULL, so crash() writes through a null pointer.

What a crash looks like in GDB

```
(gdb) run
Program received signal SIGSEGV, Segmentation fault.
0x0000555555555171 in crash (i=0x0) at crash.c:9
9    *i = 1;
(gdb) backtrace
#0  crash (i=0x0) at crash.c:9
#1  0x00005555555551a8 in f (i=0x7fffffffe0ac) at crash.c:16
#2  0x00005555555551d1 in main () at crash.c:21
(gdb) frame 1
16    crash(j);
(gdb) print j
$1 = (int *) 0x0
```

Where

Signal and exact line of the fault.

How we got here

main calls f, f calls crash.

Why

j is a NULL pointer.

Illustrative output: addresses will differ on your machine.

Demo: factorial

```c
int factorial(int n) {
  int result = 1;
  for (int i = 1; i ⩽ n; i++)
    result *= i;
  return result;
}
```

Try in GDB, input 13

```
(gdb) break factorial if n = = 13
(gdb) run
(gdb) watch result
(gdb) continue
(gdb) print result
```

The loop is correct. 12! fits in an int, but 13! = 6,227,020,800 exceeds INT_MAX (2,147,483,647).

Fix: use long long, valid up to 20!.

Intermediate GDB commands

Command	Purpose
tbreak func	One-time breakpoint
info breakpoints, delete N	List and remove breakpoints
condition N expr	Add a condition to a breakpoint
display var	Print a value at every stop
until, advance LINE	Run to a location
set var x = 5	Modify a variable
ptype var, x/8dw arr	Show a type, examine memory
layout src	Text UI with source view

Debugging OpenMP programs

```
gcc -g -O0 -fopenmp matrix_multiply.c -o matrix_multiply
OMP_NUM_THREADS=4 gdb ./matrix_multiply
```

Command	Purpose
info threads	List all threads and their state
thread / thread N	Show current / switch to thread N
set scheduler-locking on	Keep other threads stopped
print A[i][k], B[k][j], C[i][j]	Inspect per-thread values

Demo: OpenMP matrix multiplication

```
int sum = 0; // shared
#pragma omp parallel for collapse(2)
for (int i = 0; i < N; i++)
  for (int j = 0; j < N; j++) {
    sum = 0;
    for (int k = 0; k < N; k++)
      sum += A[i][k] * B[k][j];
    C[i][j] = sum;
  }
```

Compile

```
gcc -g -O0 -fopenmp matrix_multiply.c OMP_NUM_THREADS=4
.-0 matrix_multiply
```

Inspect: GDB

```
(gdb) OMP_NUM_THREADS=4 gdb ./matrix_multiply
(gdb) info threads
(gdb) thread 2
(gdb) info locals  (inside each thread, we can see
private values of the same variables)
```

Debugging tools for GPUs

NVIDIA GPUs: CUDA programs

cuda-gdb is GDB extended for CUDA kernels on NVIDIA GPUs.

```
nvcc -g -G -o app app.cu
cuda-gdb ./app
```

Use info cuda kernels and info cuda threads.

AMD GPUs: HIP programs

rocgdb is the ROCm debugger for HIP programs on AMD GPUs.

```
hipcc -g -O0 -o app app.hip
rocgdb ./app
```

Commands follow the GDB workflow you already learned.

Choose by the GPU and the programming model: CUDA on NVIDIA uses cuda-gdb, HIP on AMD uses rocgdb.

GPU debugging vs CPU debugging

Aspect	CPU debugging	GPU debugging
Threads	A few to a few dozen	Thousands, grouped into blocks (warps of 32 on NVIDIA)
Focus	One thread at a time	Pick a block and thread, e.g. cuda thread (3,0,0)
Breakpoints	Stop one thread	Stop a kernel, which halts many threads at once
Memory	One address space	Separate device memory: host code cannot read device pointers directly
Launch	Function calls run in order	Kernel launches are asynchronous, so errors can appear later
Races	Locks and atomics	Thousands of threads: __syncthreads() inside a block, atomics across blocks

GPU debugging with CUDA-GDB

Build and start

```
nvcc -g -G -o app app.cu
cuda-gdb ./app
```

Inside cuda-gdb

```
(cuda-gdb) break my_kernel
(cuda-gdb) run
(cuda-gdb) info cuda kernels
(cuda-gdb) info cuda threads
(cuda-gdb) cuda block 1,0,0 thread 3,0,0
(cuda-gdb) set cuda memcheck on
```

QUIZ

What does the bt command do?

What command sets a breakpoint at main?

What command steps into a function?

Which command continues execution until the next breakpoint?

What happens if you run a binary without -g in GDB?

Quiz answers

Q	Answer
1	Prints the call stack (backtrace)
2	break main
3	step
4	continue (or c)
5	It still runs, but GDB has no source lines or variable names: line breakpoints and print of locals fail, and backtraces show only function names and addresses

Thank you